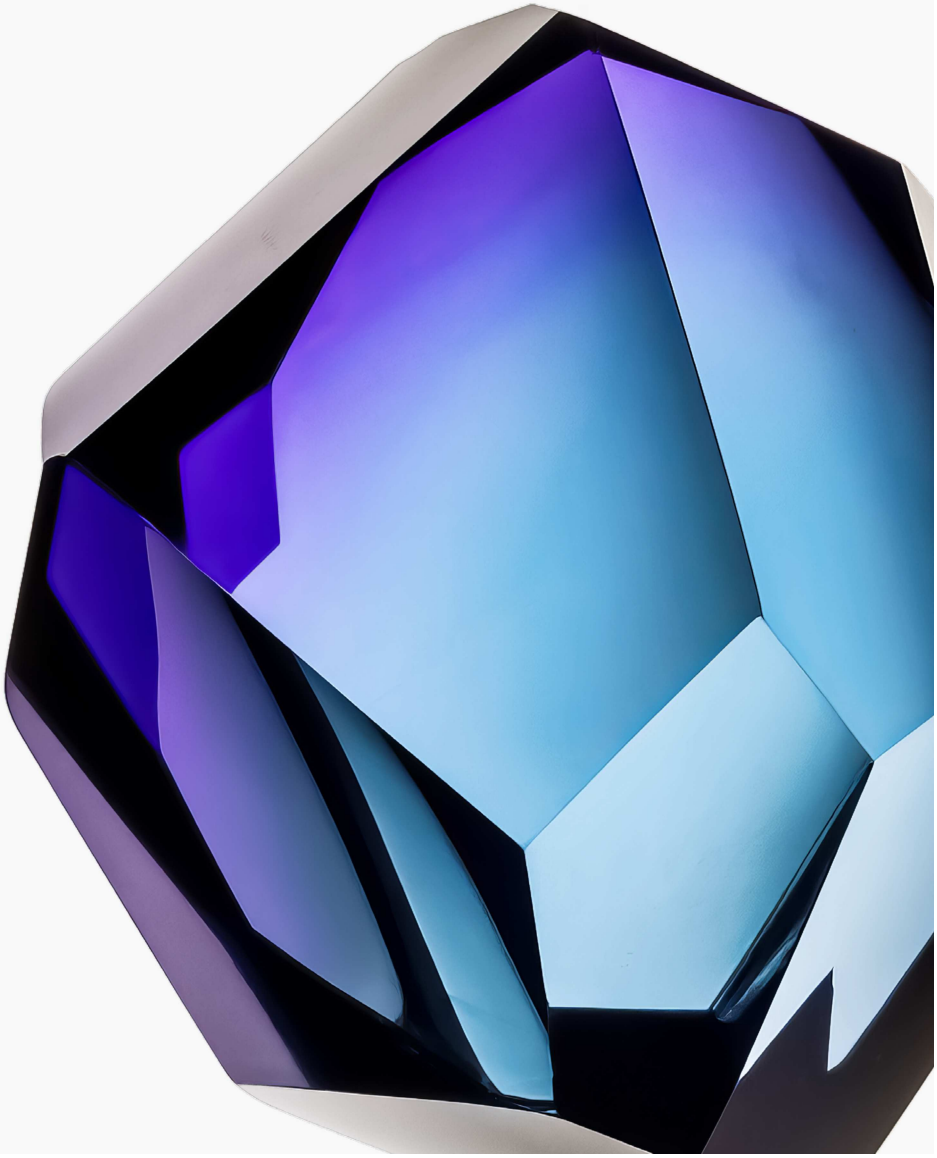
Prompt Engineering

Author: Lee Boonstra



Google

Acknowledgements

Reviewers and Contributors

Michael Sherman

Yuan Cao

Erick Armbrust

Anant Nawalgaria

Antonio Gulli

Simone Cammel

Curators and Editors

Antonio Gulli

Anant Nawalgaria

Grace Mollison

Technical Writer

Joey Haymaker

Designer

Michael Lanning

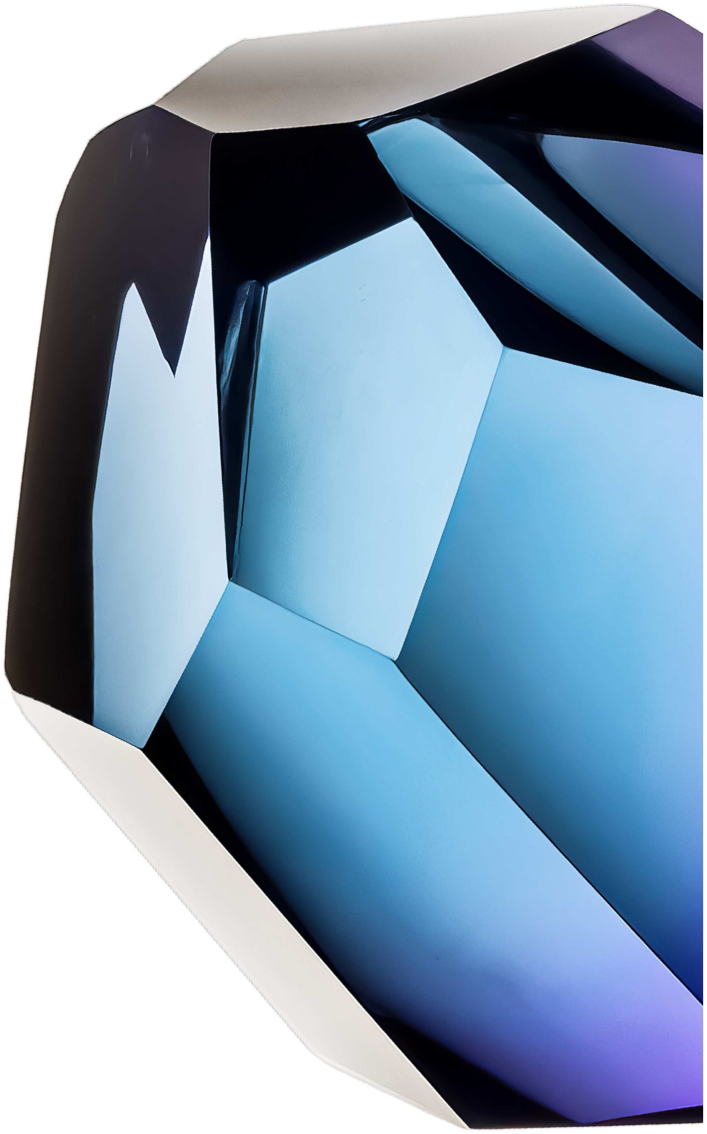


Table of contents

Introduction 6

Prompt engineering 7

LLM output configuration 8

- Output length 8
- Sampling controls 9
  - Temperature 9
  - Top-K and top-P 10
  - Putting it all together 11

Prompting techniques 12

- General prompting / zero shot 13
- One-shot & few-shot 14
- System, contextual and role prompting 17
  - System prompting 18
  - Role prompting 21
  - Contextual prompting 23



> You don't need to be a data scientist or a machine learning engineer – everyone can write a prompt.

Introduction

When thinking about a large language model input and output, a text prompt (sometimes accompanied by other modalities such as image prompts) is the input the model uses to predict a specific output. You don't need to be a data scientist or a machine learning engineer – everyone can write a prompt. However, crafting the most effective prompt can be complicated. Many aspects of your prompt affect its efficacy: the model you use, the model's training data, the model configurations, your word-choice, style and tone, structure, and context all matter. Therefore, prompt engineering is an iterative process. Inadequate prompts can lead to ambiguous, inaccurate responses, and can hinder the model's ability to provide meaningful output.

Name	1_1_movie_classification		
Goal	Classify movie reviews as positive, neutral or negative.		
Model	gemini-pro		
Temperature	0.1	**Token Limit**	5
Top-K	N/A	**Top-P**	1

Prompt	`Classify movie reviews as POSITIVE, NEUTRAL or NEGATIVE.`
	`Review: "Her" is a disturbing study revealing the direction humanity is headed if AI is allowed to keep evolving, unchecked. I wish there were more movies like this masterpiece.`
	`Sentiment:`		

Output	POSITIVE		

Table 1. An example of zero-shot prompting

When zero-shot doesn't work, you can provide demonstrations or examples in the prompt, which leads to "one-shot" and "few-shot" prompting. General prompting / zero shot

One-shot & few-shot

When creating prompts for AI models, it is helpful to provide examples. These examples can help the model understand what you are asking for. Examples are especially useful when you want to steer the model to a certain output structure or pattern.

A **one-shot prompt**, provides a *single* example, hence the name one-shot. The idea is the model has an example it can imitate to best complete the task.

A **few-shot prompt** [7] provides *multiple* examples to the model. This approach shows the model a pattern that it needs to follow. The idea is similar to one-shot, but multiple examples of the desired pattern increases the chance the model follows the pattern.

The number of examples you need for few-shot prompting depends on a few factors, including the complexity of the task, the quality of the examples, and the capabilities of the generative AI (gen AI) model you are using. As a general rule of thumb, you should use at least three to five examples for few-shot prompting. However, you may need to use more examples for more complex tasks, or you may need to use fewer due to the input length limitation of your model.

Table 2 shows a few-shot prompt example, let's use the same gemini-pro model configuration settings as before, other than increasing the token limit to accommodate the need for a longer response.

Goal	Parse pizza orders to JSON		
Model	gemini-pro		
Temperature	0.1	**Token Limit**	250
Top-K	N/A	**Top-P**	1

Prompt	`Parse a customer's pizza order into valid JSON:`
	
	`EXAMPLE:`
	`I want a small pizza with cheese, tomato sauce, and pepperoni.`
	`JSON Response:`
	```` ``` ````
	`{`
	`"size": "small",`
	`"type": "normal",`
	`"ingredients": [["cheese", "tomato sauce", "peperoni"]]`
	`}`
	```` ``` ````
	
	Continues next page...		